Real-Time Detection of ARP Spoofing Attacks in Local Area Networks

Hardik Yadav (24BCI0216), Yash Singh Jhinkwan (24BCI0123), Suhani Pasricha (23BCE2235)
Course: BCSE308L, Computer Networks

Abstract

The Address Resolution Protocol (ARP) is a fundamental component of IPv4 local area networks, responsible for mapping IP addresses to physical MAC addresses. However, ARP lacks authentication mechanisms, making it vulnerable to ARP spoofing (ARP cache poisoning) attacks. In such attacks, a malicious host sends forged ARP replies to manipulate the ARP tables of other devices on the network, enabling attacks such as Man-in-the-Middle (MITM), packet interception, and denial of service.

To address this, we propose to study the behavior of ARP spoofing attacks in a controlled local area network and develop a lightweight detection system capable of identifying malicious ARP activity in real time. A small LAN topology will first be designed using Cisco Packet Tracer to model the network architecture. The actual implementation and experimentation will then be carried out using physical or virtual machines connected on the same network. ARP spoofing attacks will be performed using Python and the Scapy packet manipulation library, allowing forged ARP packets to be generated and transmitted between hosts.

To mitigate the attack, a Python-based detection system will continuously monitor ARP traffic on the network by capturing packets and maintaining a mapping between IP and MAC addresses. Any unexpected changes in these mappings, duplicate IP-to-MAC associations, or abnormal ARP reply behavior will be treated as potential indicators of spoofing. Upon detection, the system will generate alerts and may optionally perform automated mitigation actions such as restoring the correct ARP mapping or blocking communication with the suspected attacker.

The effectiveness of the proposed system will be evaluated across varying attack scenarios, with performance benchmarked using detection accuracy, false positive rate, false negative rate, and response time. Results are expected to demonstrate reliable real-time identification of spoofing activity along with a low false-alarm rate, with the project offering practical insight into both the exploitation of a common network vulnerability and the implementation of an effective defensive countermeasure.

Keywords: ARP Spoofing, Network Security, Intrusion Detection, Man-in-the-Middle Attacks, Packet Analysis, Scapy, LAN Security

1. Introduction

Modern local area networks (LANs) rely on the Address Resolution Protocol (ARP), defined in RFC 826, to resolve a known IP address into the corresponding MAC address before a frame can be delivered at the data-link layer. ARP was designed in an era when LANs were small, trusted, and administered by a single authority, so the protocol contains no mechanism to verify that an ARP reply actually originates from the host that owns the claimed IP address. Any device on the same broadcast domain can therefore send an

unsolicited, forged ARP reply and cause other hosts to overwrite legitimate entries in their ARP cache. This weakness, commonly called ARP spoofing or ARP cache poisoning, has remained relevant for decades despite being one of the oldest known LAN-layer vulnerabilities, because switched Ethernet networks still depend on ARP by default and most enterprise and campus networks do not enable switch-level protections such as Dynamic ARP Inspection (DAI).

An attacker who successfully poisons the ARP caches of two hosts, or of a host and the default gateway, can transparently position themselves between the two endpoints and mount a Man-in-the-Middle (MITM) attack, enabling packet sniffing, session hijacking, credential theft, and selective denial of service. Because the attack operates below the transport layer and produces no direct signs at the application layer, users and even many endpoint security tools do not detect it unless the network itself is being monitored. This project studies the mechanics of ARP spoofing in a controlled LAN and develops a lightweight, host-based detection and mitigation tool that can be deployed without special switch hardware.

2. Literature Survey

Existing approaches to ARP spoofing defense fall broadly into four categories. The first is static configuration, where administrators manually bind IP-to-MAC pairs on critical hosts; this is effective but does not scale beyond a handful of machines and cannot adapt to DHCP-assigned addresses. The second is switch-assisted protection, such as Dynamic ARP Inspection and DHCP Snooping on managed switches, which validates ARP packets against a trusted binding table built from DHCP transactions; this is robust but requires enterprise-grade switching hardware that is often unavailable in smaller networks or lab environments. The third category is host- or gateway-based intrusion detection, exemplified by tools such as Arpwatch and Wireshark-based manual inspection, which passively observe ARP traffic and flag IP-to-MAC changes; these tools are widely used but are typically passive loggers rather than active, real-time alerting or mitigation systems. The fourth category includes cryptographic extensions to ARP, such as Secure ARP (S-ARP), which authenticate replies using digital signatures, but these require protocol changes on every participating host and have seen little real-world adoption because of deployment overhead.

This project builds on the passive-monitoring approach used by tools like Arpwatch but extends it with an explicit real-time alerting pipeline and an optional automated mitigation step, while remaining deployable purely in software on commodity machines. It also differs from switch-based solutions in that it does not require managed switch hardware, making it suitable for lab, campus, and small-office environments.

3. Problem Statement

Standard ARP implementations accept any ARP reply without verifying the sender's identity, which allows a malicious host on the same LAN segment to poison the ARP caches of other hosts and the gateway. Existing defenses either require specialized switch hardware, involve significant deployment overhead, or provide only passive logging without real-time alerting or automated response. There is a need for a lightweight, software-only system that can run on ordinary hosts, detect ARP spoofing activity as it happens, and take corrective action with a low false-positive rate.

4. Objectives

- Design and simulate a representative LAN topology, including a gateway and multiple end hosts, using Cisco Packet Tracer.
- Implement an ARP spoofing attack module using Python and Scapy to generate and transmit forged ARP replies between hosts on a test network.
- Implement a passive detection module that continuously monitors ARP traffic and maintains an authoritative IP-to-MAC mapping table.
- Define detection rules based on unexpected mapping changes, duplicate IP-to-MAC associations, and abnormal ARP reply frequency.
- Implement an optional automated mitigation module that restores correct ARP mappings or blocks traffic from a suspected attacker.
- Evaluate the system using detection accuracy, false positive rate, false negative rate, and response time across multiple attack scenarios.

5. Proposed Methodology

5.1 Network Topology Design

A small LAN topology is first modeled in Cisco Packet Tracer, consisting of a router acting as the default gateway, a switch, and several end hosts representing a legitimate sender, a legitimate receiver, an attacker node, and a monitoring node. This topology mirrors a typical small office or campus lab segment and is used to reason about traffic flow and the placement of the detection module before implementation on physical or virtual machines.

5.2 Attack Simulation Module

The attack module is implemented in Python using the Scapy packet manipulation library. It constructs and sends forged ARP reply packets that associate the IP address of a legitimate host, such as the default gateway, with the MAC address of the attacker's network interface. The module supports both one-way spoofing, which targets a single victim, and two-way spoofing, which poisons both the victim and the gateway simultaneously to enable full Man-in-the-Middle interception. This module is used purely to generate realistic attack traffic for evaluating the detection system in a controlled environment.

5.3 Detection Module

The detection module runs a continuous packet sniffer that inspects all ARP requests and replies observed on the network interface. It maintains an in-memory table mapping each IP address to the MAC address most recently associated with it, along with a timestamp of when that mapping was first established. Each new ARP reply is checked against this table using three rules: (i) whether the claimed IP address is already bound to a different MAC address, indicating a possible cache-poisoning attempt; (ii) whether a single IP address is being associated with multiple MAC addresses within a short time window, indicating duplicate IP-to-MAC associations; and (iii) whether the rate of ARP replies for a given IP address significantly exceeds the expected baseline, indicating abnormal ARP reply behavior. A packet that triggers any of

these rules is flagged as suspicious and logged with the source and destination addresses, the claimed mapping, and a timestamp.

5.4 Mitigation Module

When the detection module raises an alert, the system generates a human-readable notification and can optionally trigger an automated response. Two mitigation actions are supported: restoring the correct IP-to-MAC mapping by issuing a corrective ARP announcement to the local network, and blocking further communication with the suspected attacker's MAC address at the host firewall level. Automated mitigation is configurable so that it can be disabled during evaluation, allowing the detection accuracy to be measured independently of the mitigation step.

6. System Architecture

The system is organized into four logical components deployed across the simulated LAN: an Attacker Node running the Scapy-based spoofing module, one or more Victim Nodes representing ordinary end hosts and the gateway, a Monitoring Node running the detection and mitigation modules in promiscuous packet capture mode, and a shared Ethernet switch connecting all nodes. The Monitoring Node observes all ARP traffic traversing the switch, applies the detection rules described in Section 5.3, and communicates any alerts or mitigation actions back onto the network. This separation keeps the attack, detection, and mitigation logic independent, which simplifies controlled testing of each component and allows the detection accuracy to be measured against a known ground truth of injected attacks.

7. Implementation Plan

The implementation uses Python 3 as the primary language, with Scapy for packet crafting, sniffing, and injection. Cisco Packet Tracer is used for initial topology design and validation of expected traffic flow. The experimental network is realized using either physical machines on an isolated switch or virtual machines (for example, under VirtualBox or VMware) configured on a shared virtual network, so that ARP traffic between hosts can be generated and observed safely without affecting a production network. Wireshark is used alongside the custom detection module during development to independently verify that captured ARP packets and flagged events correspond to the traffic actually being generated by the attack module.

The detection module's core logic follows a straightforward sniff-check-alert loop: the module opens a live capture filtered to ARP traffic, extracts the sender IP and sender MAC from each packet, compares the pair against the current mapping table, updates or flags the entry accordingly, and writes any alert to a log file with a timestamp. This loop is designed to run continuously with minimal CPU and memory overhead so that it can operate alongside normal host activity without noticeably affecting network performance.

8. Evaluation Plan and Expected Results

The system will be evaluated under several attack scenarios, including single-victim spoofing, gateway impersonation, and full-duplex Man-in-the-Middle spoofing, each repeated across multiple trials with

varying background traffic levels. For each scenario, the following metrics will be recorded:

- Detection accuracy — the proportion of injected spoofing attempts correctly identified by the system.
- False positive rate — the proportion of legitimate ARP updates (such as those following normal DHCP lease renewal) incorrectly flagged as spoofing.
- False negative rate — the proportion of actual spoofing attempts that go undetected.
- Response time — the time elapsed between the transmission of a forged ARP reply and the system raising an alert or applying mitigation.

The system is expected to achieve high detection accuracy and low false-positive and false-negative rates under typical LAN conditions, with response times on the order of a few seconds, since the detection logic operates directly on live traffic rather than periodic polling. Performance is expected to degrade gracefully, rather than fail outright, as background network traffic increases, since the detection rules depend on IP-to-MAC consistency rather than overall traffic volume.

9. Conclusion and Future Scope

ARP spoofing remains a practical and easily exploitable vulnerability in modern LANs because the protocol was never designed with authentication in mind. This project demonstrates both the ease with which ARP cache poisoning can be carried out using freely available tools such as Scapy, and the feasibility of detecting and mitigating such attacks in real time using a lightweight, purely software-based approach that does not depend on specialized switch hardware. The resulting system is intended as a practical proof of concept for real-time ARP spoofing detection suitable for small networks, labs, and educational environments.

Future work could extend the detection logic to wireless LANs, where ARP spoofing interacts with additional layer-2 considerations; integrate the detection module with Software-Defined Networking (SDN) controllers to enable centralized, network-wide mitigation; and explore machine-learning-based anomaly detection to improve accuracy on larger, higher-traffic networks where simple rule-based thresholds may need to be replaced with adaptive models.

References

[1] D. C. Plummer, "An Ethernet Address Resolution Protocol," RFC 826, Internet Engineering Task Force, 1982.

[2] Scapy Project, "Scapy Documentation," https://scapy.readthedocs.io/.

[3] Cisco Systems, "Cisco Packet Tracer," https://www.netacad.com/courses/packet-tracer.

[4] L. L. Whitesell and V. Bahl, "Arpwatch: Ethernet/FDDI station activity monitor," Lawrence Berkeley National Laboratory.

[5] Wireshark Foundation, "Wireshark User's Guide," https://www.wireshark.org/docs/.